Team Name	teamMeowMan()	Team ID	TM-75635E4A444
Team Leader	Shaunak Choudhury	Members	Pranjal Kumar, Malika Farah
Problem	Theme 6: AI You Can Actually Trust. Problem 9: Catching an Agent Before It Acts on a Bad Idea		

Aegis: checking an agent's tool calls against the database before they run

1. Problem & Theme chosen

We chose Problem 9. The agent mistakes that worry us most are the quiet ones: cancelling the customer's other pending order, or exchanging a keyboard for the wrong variant. The API accepts these because the request is valid, and nobody notices until the money has moved.

2. Understanding of the challenge

Every argument of a tool call raises three questions:

- **Is it legal?** The record exists, belongs to the logged-in user, and its state allows the action.
- **Where did it come from?** The user, a trusted tool output, or nowhere.
- **Is it what the user meant?**

The first two can be checked exactly against the database and a log of the chat. The third is the hard one, and recent provenance-based guards [1-4] do not target it: if the agent picks the wrong one of the user's own orders, that ID is real, owned and was shown earlier, so it passes. An LLM judge struggles too, since an ID like `#W2378156` means nothing without a lookup, and running a large model on every step is wasteful when 68% of τ²-bench retail gold actions are plain lookups.

Pilot. We made 1,000 broken copies of the 174 correct write calls in τ²-bench retail [5], each with one argument changed: a made-up ID, another customer's order, an order in the wrong state, a changed value, or a different record of the same user. 252 ran with no error. Ownership/state rules plus exact provenance caught all of them except same-user swaps: 157 of 239 (66%) got through, while no correct call was blocked (Fig. 1). These are exactly the "plausible but ungrounded" actions of Problem 9.

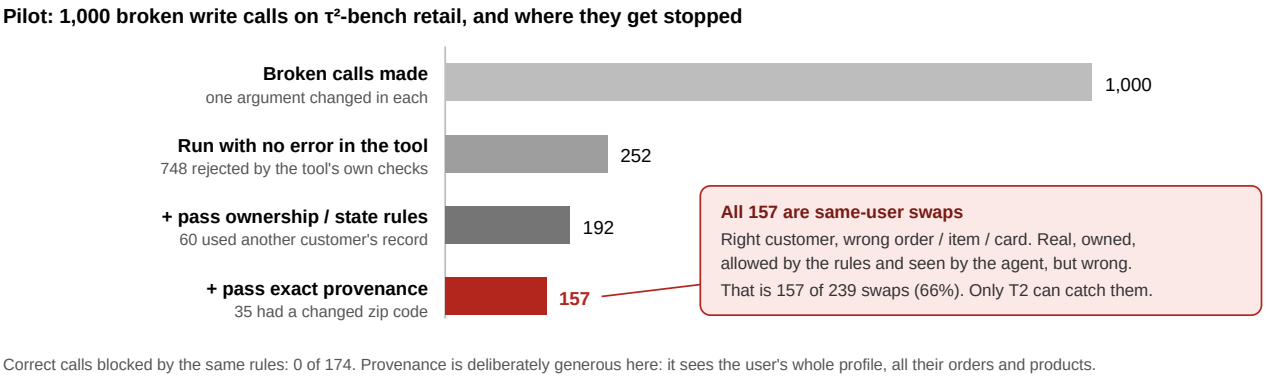


Figure 1. Pilot result. Everything except same-user swaps is caught by checks that need no model at all, which is why T2 focuses on intent.

3. Proposed approach

Aegis wraps every tool of a LangGraph agent and checks each call before it runs, cheapest check first, stopping at the first one that can decide (Fig. 2).

- **T0, schema and existence (no LLM).** Pydantic types, and every ID looked up in a read-only database snapshot.
- **T1, rules and provenance (no LLM).** Per-tool YAML rules such as `order.user_id == session.user` or `order.status == "pending"`, and a ledger of where every value was seen. Read-only calls stop here.
- **T2, look up and compare (our main idea).** A small local model (Qwen3-8B, 4-bit, one consumer GPU) turns the user's request into an intent frame such as `switch=clicky, backlight=RGB or none`, with each slot quoting the user's words. Every ID in a write call is replaced by its database attributes and compared with the frame, in plain code where possible; only unclear slots go to the model. If the user never specified something the call needs (e.g. which card to refund to), T2 asks the user instead of guessing.
- **T3, careful review.** A frontier model or the user, only when the risk score (irreversible, money involved, T2 unsure) is high.

A blocked call returns its reason to the agent, so it can fix the call and retry (Fig. 3).

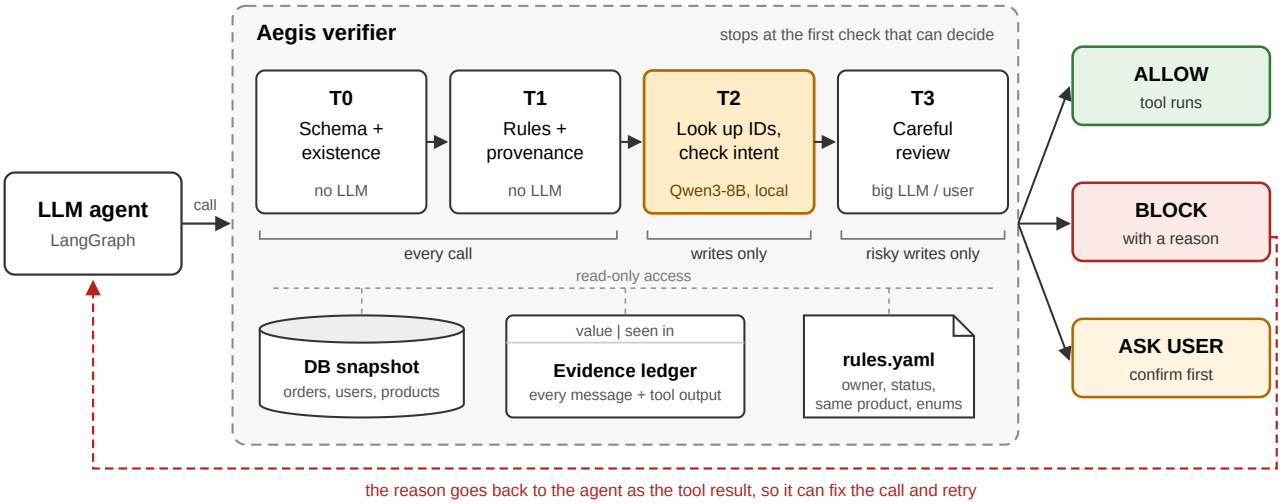


Figure 2. Aegis architecture. Reads never reach a model. Writes reach T2, and only risky or unclear writes reach T3 or the user.

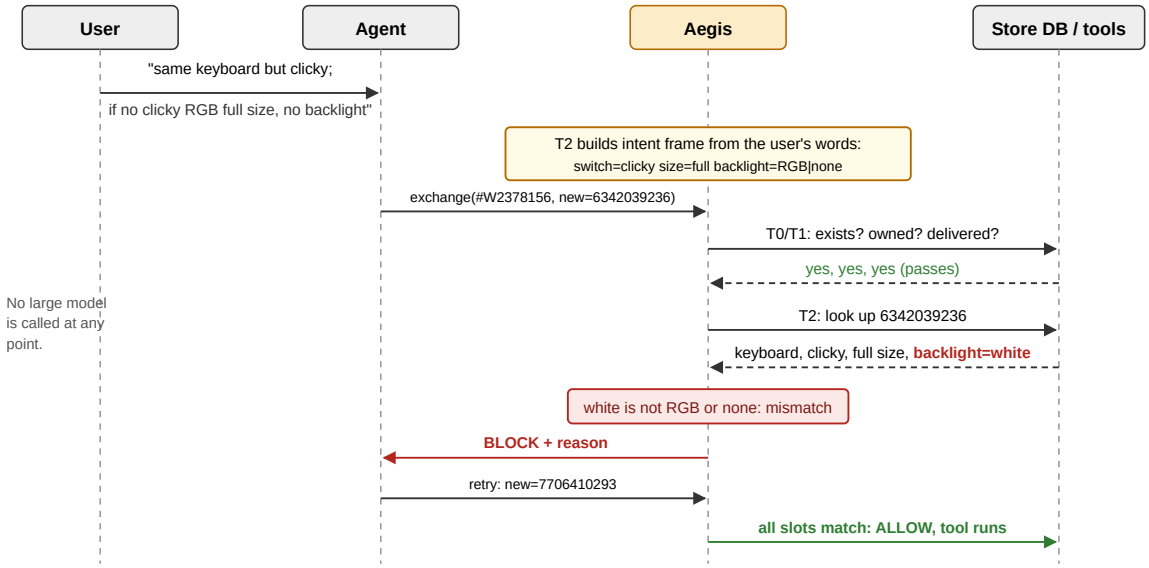


Figure 3. τ²-bench retail task 0, using the real database records. The wrong item is real, in stock, the same product and was shown to the agent, so T0 and T1 let it through. Looking it up shows the wrong backlight. The retry matches the benchmark's gold action.

4. Minimum Viable

Requirement in the problem statement	What we will demo
Verify at least one class of action against a verifiable source	All 7 write tools of τ²-bench retail, checked against the database snapshot and the store policy.
Catch at least one deliberately bad / hallucinated action	A live agent run where the wrong keyboard variant is blocked and then fixed (Fig. 3), and the 1,000 broken calls replayed in a small Streamlit viewer showing the verdict and reason for each.

5. Stretch ambitions

- **Fast vs careful routing by risk** (the problem's first stretch goal): T2 on Qwen3-8B, T3 on a frontier model. We will plot how much recall we keep as fewer calls are sent to the big model.
- **Red-teaming suite:** the broken-call generator run on τ²-bench airline, plus about 60 tricky user scripts: vague references ("my last order"), changing their mind halfway, acting on someone else's account, and instructions hidden in product descriptions.

6. Novelty / differentiation

CaMeL [1], Agent-Sentry [2], ProvenanceGuard [3] and PACE [4] track where values come from, mainly to stop prompt injection; our pilot shows a class of mistakes this cannot see. What is new:

- Looking up IDs before judging, so most intent checks are simple comparisons with no model call.
- Labelled test cases for free, by mutating a benchmark's gold actions.
- Block reasons fed back to the agent, measured by whether task success goes up.

7. Evaluation plan

Setting	Data	Metrics
Offline	Broken-call sets for retail and airline (5 random seeds, 95% bootstrap confidence intervals)	Recall per mistake type, false blocks on correct calls, added latency (median and p95), cost per 1,000 calls, share of calls reaching each tier
Online	Full τ^2-bench runs with its user simulator; Qwen3-8B and one frontier model as the agent	Task success rate, wrong writes (database diff against gold), tasks we wrongly interrupt

Baselines: no guard; the tools' own checks; provenance only; a frontier LLM judge on the full transcript. **Ablations:** no ID lookup, no intent frame, always-small, always-large model. **Our bar, fixed now:** beat the LLM judge on same-user swaps, block at most 2% of correct calls, add under 5 ms to reads, cost less than the judge, and not lower task success.

8. Feasibility and Risks

Team. B.Tech CSE students at VIT-AP. Shaunak and Pranjal won Recon 2026 (CTF, 1st of 100 teams) and placed in the top 14 of 6,700+ at Google Solution Challenge 2026.

- **Shaunak Choudhury (leader).** Support Engineer at Vapi, where he debugs failed LLM tool calls in production voice agents. Built Lunetra, a support platform where agent actions need approval before they run, which is the same problem space as Aegis. Owns the agent harness, tool wrapper, ledger, intent frame and router.
- **Pranjal Kumar.** Penetration-testing intern; wrote 18 Python detectors that classify mistakes in chess games for Zugli. Owns the broken-call generator, red-team suite and demo viewer.
- **Malika Farah.** CGPA 9.40; built Isolation Forest fraud detection with a hash-chained ledger (CyberSentinel), bias-audit metrics (FairLens) and role-based access rules (CEMS). Owns the YAML rules, baselines and evaluation statistics.

Week	Milestone
Nov 10 - 16	Retail tools wrapped, ledger, T0 and T1 rules for all write tools
Nov 17 - 23	Intent frame, ID lookup, T2; first offline numbers
Nov 24 - Dec 1	Router, T3, block reasons, airline rules; midway report
Dec 2 - 8	Online runs, LLM judge baseline, ablations
Dec 9 - 14	Red-team scripts, demo viewer, practice, two days of buffer

What could go wrong. A wrong intent frame could block good calls, so an unsure frame asks the user instead. Rules are hand-written per domain; we will report the airline porting effort. Broken copies are not real hallucinations, so the online runs matter. Running the agent and T2 locally keeps costs low.

References. [1] Debenedetti et al., CaMeL, arXiv:2503.18813, 2025. [2] Sequeira et al., Agent-Sentry, arXiv:2603.22868, 2026. [3] She et al., ProvenanceGuard, arXiv:2607.01236, 2026. [4] Li et al., PACE, arXiv:2610.01349, 2026. [5] Barres et al., τ^2-Bench, arXiv:2506.07982, 2025.